

Raíz Andina – solución digital integral para tienda naturista

Demo funcional que sostiene la propuesta comercial: no es una página web, es el sistema que reduce pérdidas por quiebre de stock y ahorra el tiempo administrativo.

30 módulos 512 pruebas 1 dependencias 400 productos

CONTENIDO

Arrancar

Dos canales, un solo stock

 La ganancia

El catalogo real del negocio

Respaldo

 Dos papeles: el dueño y el mostrador

Guion de la demo (12 minutos)

 Acto 1 – La vista del cliente (3 min)

 Acto 2 – El pedido y el descuento de stock (4 min)

 Acto 3 – La IA como diferenciador (3 min)

 Cierre (2 min)

Arquitectura

El catálogo de 400 productos

Alta de productos

Edición del catálogo

Checkout peruano

Comprobantes electrónicos

Acceso al panel

Las tres capas del asesor

Imágenes: tres niveles de respaldo

La decisión de diseño que sostiene la demo

Activar la redacción con Claude (opcional)

API

Lo que falta para producción

Arrancar

```
copy .env.example .env      # y pon ADMIN_PASSWORD (ver «Acceso al panel»)
node gen-ubigeo.mjs          # compacta el ubigeo del INEI (una sola vez)
node gen-catalogo.mjs       # arma el catálogo de 400 productos de ejemplo
node gen-imagenes.mjs       # dibuja las ilustraciones (por SKU y de respaldo)
node importar-fotos.mjs     # importa las fotos propias de la carpeta img/
node traer-fotos.mjs        # rellena las que falten desde bancos libres (internet)
node db.js --reset          # carga la tienda con 400 productos
npm start                   # http://localhost:4000
```

Para poner las fotos del negocio: se dejan en la carpeta `img/` de la raíz, con el **nombre del producto** como nombre de archivo, y se ejecuta `node importar-fotos.mjs`. No hace falta que el nombre sea exacto — tolera tildes, mayúsculas, plurales y erratas (`gaviola en capsula.jfif` encuentra *Graviola en cápsulas*). Con `--probar` muestra el emparejamiento sin escribir nada.

Una foto propia manda sobre cualquier otra: `traer-fotos.mjs` no la pisa ni con `--forzar`, y `db.js --reset` la conserva. La carpeta `img/` está **fuera de** `public/`, así que los originales no se sirven por HTTP.

`traer-fotos.mjs` es lo único que necesita internet, y una sola vez: las fotos quedan en `public/img/fotos/`. Después la tienda funciona desconectada.

Dos canales, un solo stock

El negocio vende por **dos** lados y los dos descuentan del **mismo** inventario:

	CÓMO ENTRA	ESTADO INICIAL
Tienda web	el cliente compra en <code>http://localhost:4000</code>	<code>pendiente</code> , hay que despachar
Mostrador	la encargada cobra desde el panel	<code>entregado</code> , se lo llevó puesto

Hasta ahora el sistema solo sabía registrar la venta de la web. Lo que se vendía de frente — que en un puesto de mercado es casi todo— no bajaba del inventario, así que **el stock del panel era mentira a media mañana**.

La sección **Venta en el local** es la caja del puesto, y es lo primero del panel porque es lo que se usa con un cliente delante. Se busca el producto, se agrega, se cobra:

- Al buscar se muestra **el stock de cada uno**, porque la pregunta del mostrador no es «cuánto cuesta» sino «¿me queda?». El agotado no se puede agregar y lo dice, en vez de dejar un botón muerto.
- El carrito **nunca promete lo que no hay**: el `+` se apaga al llegar al stock, y si aun así se intenta, el servidor devuelve cuántas quedan de cada producto antes de emitir nada — con el cliente delante hay que poder decirle «de ese me queda uno».

- **El documento es opcional.** Pedirle el DNI a quien compra muña de S/ 9 es perder la venta: sin documento sale boleta a nombre del mostrador. Si se escribe un RUC de 11 dígitos aparece sola la razón social y sale factura.
- Si el cobro falla, **el carrito se queda ahí.** Una venta a medias no se borra mientras el cliente espera el vuelto.

El tablero separa los dos canales —«S/ 54.80 en el local · S/ 0.00 por la web»— y eso es lo que permite comprobar que cuadra: si se vendieron 3 en el local y 2 por la web, el inventario tuvo que bajar 5. En el kardex cada movimiento dice de dónde vino:

Mostrador RA-... o Pedido RA-... .

La ganancia

El panel muestra la **ganancia del día** y la de cada venta, y solo al dueño: se calcula del costo, y el costo es suyo.

Se calcula con **el costo del momento en que se vendió**, no con el de hoy. El costo cambia cada vez que sube el proveedor, y recalcular la ganancia del mes pasado con el costo de esta semana da una cifra que no ocurrió nunca. Por eso cada línea de pedido guarda su `costo_unit` congelado.

El catalogo real del negocio

La tienda arranca con 400 productos de ejemplo, que son relleno para que el catalogo se vea lleno y el asesor tenga con que trabajar. Para cargar los del negocio:

```
node db.js --vacio                # 1. vaciar el de ejemplo
node importar-catalogo.mjs lista.csv --probar    # 2. ver que pasaria
node importar-catalogo.mjs lista.csv            # 3. cargarlo
node importar-fotos.mjs              # 4. las fotos
```

El paso 1 no es opcional, y no es evidente por que. El nombre de un producto es unico, asi que los 400 de ejemplo **ocupan los nombres de verdad**: si el negocio vende «Maca Negra en polvo» y el ejemplo ya la tiene, el producto real se rechaza. Darlo de baja no libera el nombre, porque la baja no borra la fila. `--vacio` borra los productos y conserva el resto de la base — usuarios, pedidos, comprobantes; un producto que ya tuvo una venta no se borra (dejaría una venta sin producto), se conserva de baja y marcado `(retirado)` .

El archivo se puede armar en Excel y guardar como CSV. `plantilla-catalogo.csv` es el modelo. Solo cuatro columnas son obligatorias — `nombre` , `categoria` , `presentacion` , `precio` — y el resto rellenan la ficha: `costo` , `stock` , `stock_min` , `origen` , `beneficios` , `uso_tradicional` , `etiquetas` , `sku` , `descripcion` , `imagen` .

El lector aguenta el archivo tal como sale del Excel peruano: separador `;` o coma, coma decimal (`28,50`), el `S/` delante, tildes y mayusculas en las cabeceras (`Categoría` , `Stock min`), el BOM invisible y los saltos CRLF. `--probar` no escribe nada; sin `--probar` , **antes de tocar el catalogo se hace un respaldo.**

Las filas con problemas se rechazan **una por una, diciendo el numero de linea del archivo** y el motivo con las palabras del propio sistema: precio en cero, costo por encima del precio, nombre repetido. Las buenas entran igual, así que se corrigen las malas y se vuelve a pasar el mismo archivo – lo que ya entro se rechaza solo por nombre repetido y no se duplica.

El importador entra por `POST /api/productos`, **la misma puerta que el panel**, por eso necesita el servidor encendido. Las reglas de un producto viven en un solo sitio: lo que el panel rechaza, esto lo rechaza, sin escribir la regla dos veces. El SKU lo asigna el sistema por categoria (`SUP-001` , `HIE-001`) si el archivo no trae uno, y el stock inicial entra como movimiento de kardex, no como un numero puesto a dedo.

Mientras no haya fotos, cada producto usa la ilustracion que le corresponde por presentacion y categoria. Con `node importar-catalogo.mjs --bajar-demo` se saca de la tienda el relleno que quede activo, sin borrarlo.

Respaldo

Todo el negocio vive en un archivo, `data/tienda.db`. **El respaldo se hace solo:** uno por día, en cuanto se enciende la laptop y se arranca el servidor. No se programa a una hora porque el puesto apaga la máquina al cerrar y una tarea de madrugada nunca correría.

```
node respaldo.mjs                                # copiar ahora
node respaldo.mjs --listar                        # ver qué copias hay
node respaldo.mjs --restaurar tienda-2026-09-11.db # volver a una (servidor parado)
```

El panel lo muestra en el bloque **Respaldo**, y se pone en rojo si hoy no se hizo ninguno. Un respaldo que hay que ir a comprobar es un respaldo que nadie comprueba.

Apúntalo fuera del disco. Por defecto las copias quedan en `data/respaldos/`, al lado de la base: eso salva de un borrado por error, pero no de que se lleven la laptop. Con `RESPALDO_DIR` va a un pendrive o disco externo, y el panel deja de avisar:

```
RESPALDO_DIR=E:
espaldos npm start
```

Se guardan las últimas **14** copias (`RESPALDO_DIAS` lo cambia); cada una pesa unos 270 KB. La copia se hace con `VACUUM INTO`, no copiando el archivo: la base corre en modo WAL y copiarla a pelo da una base **a la que le faltan las últimas ventas y que abre sin quejarse**. Antes de guardarse, cada copia se abre y se le cuentan las filas – un respaldo ilegible se ve igual que uno bueno hasta el día que hace falta.

Restaurar deja la base anterior guardada como `antes-de-restaurar-...` en la misma carpeta, por si el que restauró se equivocó de copia.

Node 22.5+ (usa `node:sqlite` nativo). Una sola dependencia, `qrcode`, que dibuja el QR de los comprobantes: **corre npm install antes de salir hacia el local**, porque allá puede no haber internet. Con eso instalado todo funciona sin conexión, que es exactamente lo que hace falta para demostrar en la laptop del cliente.

RUTA	QUIÉN ENTRA
<code>http://localhost:4000</code>	Pública – la portada: historia, orígenes y proceso
<code>http://localhost:4000/tienda</code>	Pública – la tienda: asesor, catálogo y carrito
<code>http://localhost:4000/creditos.html</code>	Pública – atribución de las fotos
<code>/panel</code> (o <code>/admin.html</code>)	Personal. Pide login
<code>/imagenes.html</code>	Personal. Hoja de contactos de las ilustraciones
<code>/comprobante.html</code>	Las dos cosas: la tienda entra con <code>?id=</code> y sesión; el comprador con <code>?codigo=</code> y los últimos 4 del teléfono

La tienda no enlaza al panel por ningún lado. Un cliente no debe encontrarlo por curiosidad. Los atajos `/panel` y `/entrar` existen para dictarlos por teléfono.

Acceso al panel: se entra con **correo y contraseña**. El servidor crea el primer acceso la primera vez que arranca, cuando todavía no hay ningún usuario. **Ninguna instalación queda con una clave conocida:** ya no existe una clave de fábrica escrita en este README.

- 1. Con clave propia (lo normal).** Antes del primer arranque se copia `.env.example` a `.env` y se pone la clave en `ADMIN_PASSWORD` (y el correo en `ADMIN_EMAIL`, si no es el de `tienda.config.js`). `entorno.js` lee el `.env` solo al arrancar; lo que ya venga definido en la consola manda sobre el archivo.
- 2. Sin clave propia.** Si `ADMIN_PASSWORD` falta, o se dejó el `cambia-esta-clave` del ejemplo, el servidor genera una clave al azar de 12 caracteres y la muestra **una sola vez** en la consola. Solo se guarda su hash: hay que anotarla en ese momento.

Acceso al panel creado:
 correo: hola@raizandina.pe
 contrasena: (la de tu .env, o la generada al azar)

Si se pierde, o para cambiarla cuando quieras:

```
node clave.mjs hola@raizandina.pe miClaveSegura      # cambiar contraseña (mín. 8)
node clave.mjs --correo admin dueno@raizandina.pe    # cambiar el correo
node clave.mjs --nuevo rosa rosa@raizandina.pe "Rosa Q." claveDeRosa  # acceso nuevo
node clave.mjs --rol rosa admin                      # ascender a dueña
node clave.mjs --listar
```

Dos papeles: el dueño y el mostrador

	ADMIN – EL DUEÑO	VENDEDOR – EL MOSTRADOR
Pedidos: ver, preparar, anular	sí	sí

	ADMIN — EL DUEÑO	VENDEDOR — EL MOSTRADOR
Stock: ingresar mercadería, ajustar el mínimo	sí	sí
Comprobantes, clientes, calendario de ventas	sí	sí
Costo y margen de cada producto	sí	no
Valor del inventario a costo	sí	no (ve las unidades)
Cambiar precios · dar de alta o de baja fichas	sí	no
Bitácora de cambios de ficha · respaldo	sí	no

Que quien atiende no vea el costo no es desconfianza: **el margen es la negociación del dueño con su proveedor**, y no tiene por qué estar en la pantalla del mostrador, donde cualquiera se asoma.

Un acceso nuevo nace `vendedor` si no se dice otra cosa — es más fácil ascender a alguien que descubrir que llevaba meses viendo los márgenes.

El corte está **en el servidor, no en los botones**. El panel esconde lo que el rol no puede usar, pero eso es cortesía: esconder un botón evita el error de buena fe, no la curiosidad. Al mostrador el catálogo le llega **sin la columna `costo`**, y `PATCH /api/productos/:id` con un precio responde 403 sin escribir nada — ni el precio ni lo que venía junto en la misma petición.

El rol se lee de la tabla en cada petición, no de la cookie: degradar a alguien tiene efecto en su siguiente clic, sin cerrarle la sesión.

`ADMIN_PASSWORD` solo se usa en el primer arranque. Cambiarla después en el `.env` no cambia la clave del panel: para eso está `node clave.mjs`. Quien use `importar-catalogo.mjs` sí la necesita en el `.env`, porque entra al panel con ella.

Por qué se conserva el usuario corto además del correo: es lo que firma el kardex y la bitácora. En un movimiento de stock se lee mejor `Ingreso de mercadería (rosa)` que `(rosa@raizandina.pe)`. Para entrar sirven los dos, pero el correo es el camino natural cuando haya recuperación de contraseña.

Antes de una demo real, edita `tienda.config.js`. Ahí están dirección, horario, zonas de reparto en Lima y **envío a provincia**, medios de pago, datos de facturación (RUC), escala de descuento por volumen, condiciones de mayorista y política de devolución. El asesor responde con esos datos: cámbialos por los del cliente y empieza a hablar de SU negocio sin tocar código.

Guion de la demo (12 minutos)

Acto 1 — La vista del cliente (3 min)

Abre la portada (/) en pantalla completa.

1. **La portada cuenta, la tienda vende.** La portada es solo informativa: baja por la historia –orígenes, pisos ecológicos, cómo comprar– sin botones de compra. A la tienda se entra por el ítem **Tienda** del menú. El carrito es el mismo en las dos páginas: lo que se agrega sigue ahí al volver.
2. **El origen es el argumento.** Cada tarjeta dice "*Meseta de Bombón, Junín*", no solo "Maca". Y trae el uso tradicional. Eso es lo que una farmacia no puede copiar.
3. **Filtra por categoría y busca "muña":** responde al instante, sin recargar.
4. Nota que cada tarjeta dice solo *Disponible* o *Agotado*. **Nunca una cifra.** Nadie compra lo que no hay, y la competencia que entra a mirar no se entera de cuánto tiene usted en el almacén. Las cantidades las ve el dueño en el panel.

Acto 2 – El pedido y el descuento de stock (4 min)

Antes de este paso, ten el panel abierto en otra pestaña.

1. Agrega **Propóleo en gotas**. En la tienda solo dice *Disponible*; en el panel tiene stock 4 y mínimo 10. **El cliente no ve el inventario; el dueño, sí.**
2. Pon 3 unidades, confirma con datos de cliente. Sale el código **RA-...**. **Anótalo: es el que cierra el Acto 3.**
3. **Cambia a la pestaña del panel.** Sin tocar nada:
 - Ventas de hoy subió.
 - El pedido aparece con nombre, teléfono, dirección y nota.
 - *Reposición urgente* ahora marca Propóleo en **1 de 10**.
 - En movimientos de inventario: **-3 Propóleo · Pedido RA-... · queda 1**.

Frase para el cliente: "*Esto es lo que hoy le toma media hora de cuaderno. Acá pasó solo, en el mismo segundo en que el cliente pagó.*"

1. Vuelve a la tienda e intenta comprar 5 propóleos. El sistema lo bloquea y dice qué producto no alcanza, sin dar la cifra. **Ahí está la pérdida que se evita:** vender lo que no existe.
2. En el panel, pulsa **+19** en Propóleo → repuesto, con su movimiento registrado.
3. **Haz un segundo pedido** –cualquier cosa, una unidad– y **anula ese:** el stock vuelve solo al inventario y la nota de crédito **BC01-...** se emite sin que nadie la pida.

No anules el del paso 2. Con la base recién reiniciada es el único que existe, y es el que vas a pegar en el asesor para cerrar el Acto 3. Anulado, el remate responde "*figura como anulado*" en vez de "*está registrado y hoy mismo te contactamos*": cierto, pero flojo para rematar.

Acto 3 – La IA como diferenciador (3 min)

En el asesor –la primera pantalla de **/tienda** –, escribe "**no puedo dormir y ando con mucho estrés**".

- Recomienda Valeriana + Pasiflora, Manzanilla y Graviola, y **resalta esas tarjetas** en el catálogo.

- **El punto que cierra la venta:** *"La IA solo puede recomendar lo que está en su almacén ahora mismo. Si algo se agota, deja de ofrecerlo automáticamente y avisa."*

Luego escribe **"estoy embarazada, qué me recomienda"**. El asesor **se niega a recomendar** y deriva a un profesional.

"Esto lo protege a usted. Un chatbot común le recomienda cualquier cosa a una gestante y le trae un problema legal. Este sabe cuándo callarse."

Remátalo con dos frases como las dice la gente, sin palabra técnica: **"mi bebé tiene tos"** y **"tengo el azúcar alta"**. Las dos derivan igual, y la respuesta no termina en un portazo: ofrece **agendar una atención con la dueña** en el local.

"Un bot que le recomienda miel a un bebé o yacón a un diabético es la prueba perfecta en una inspección. Este lo convierte en una visita a su local, que es justo donde usted le gana a la competencia."

Prueba también **"me lo dejas en 20 soles"** (responde con la política de descuento por volumen, no con un precio inventado) y **"aceptan yape"**.

El momento mayorista – el de mayor impacto económico. Escribe:

"cuánto me sale 50 bolsas de maca negra"

El asesor aplica el 20 % por volumen y da el total... **y dice si se lo puede entregar todo hoy**. Con 42 en almacén, no: *"una parte te la entrego de una vez y el resto llega en 3 a 5 días hábiles. Te confirmo por WhatsApp cuántas salen ya"*. Nunca dice la cifra.

"Fíjese en lo que acaba de pasar: le cotizó, le dio el descuento que corresponde y le dijo si se lo entrega todo hoy, sin mostrarle a la competencia cuánto tiene usted. Hoy contesta eso revisando el depósito y llamando al proveedor. Y si se equivoca, se equivoca en el pedido más grande del mes."

Y el remate, el que amarra los tres actos: pega el **código del pedido del paso 2 del Acto 2** –el que anotaste, no el que anulaste– seguido de *"no llega"*. El asesor lo busca en la base y responde con su estado, fecha y monto reales:

"Encontré tu pedido RA-..., del ..., por S/ 64.50: está registrado y todavía no sale del local. Hoy mismo te contactamos para coordinar la entrega."

El teléfono **no** lo dice, y conviene señalarlo en voz alta: el código es una credencial débil, así que el asesor informa del pedido sin soltar datos personales. Para ver la dirección o descargar el comprobante hacen falta además los últimos cuatro dígitos del teléfono.

"No es un chat pegado al costado de la web. Es el mismo sistema respondiendo."

Cierre (2 min)

Tres números, del panel:

ANTES	CON EL SISTEMA
Stock en cuaderno, revisado los domingos	Actualizado en el segundo de la venta
Se entera del quiebre cuando el cliente ya se fue	Alerta al llegar al mínimo, con cantidad sugerida
No sabe qué producto le deja más	<i>Más vendidos</i> y valor de inventario a costo

Arquitectura

tienda.config.js	Datos y políticas del negocio <- EDITAR ANTES DE LA DEMO
db.js	Esquema SQLite + catálogo semilla + migraciones
server.js	HTTP + API REST + archivos estáticos (sin framework)
auth.js	Sesiones del panel (scrypt, node:crypto)
clave.mjs	Alta de usuarios y cambio de clave desde consola
intenciones.js	Capas 1 y 2 del asesor: cotización y preguntas de negocio
asesor.js	Capa 3 del asesor: productos + redacción con Claude
comprobantes.js	Emisión de boletas, facturas y notas de crédito + XML UBL
comprobante-pdf.js	Dibuja la boleta o factura como PDF
pdf.js	Escritor de PDF mínimo, escrito a mano y sin dependencias
catalogos-sunat.js	Códigos de los catálogos 01, 02, 03, 05, 06, 07 y 09
importe-letras.js	"SON: CIENTO VEINTICINCO CON 80/100 SOLES"
documentos.js	Validación de DNI, RUC (módulo 11), correo y teléfono
ubigeo.js	Departamento/provincia/distrito + zona y costo de envío
limites.js	Cuotas por IP (token bucket en memoria)
markdown.mjs	Markdown a HTML, lo justo que usa este README
manual.mjs	Viste este README para imprimirlo: portada, índice y hoja A4
ilustraciones.mjs	Qué lámina le toca a cada producto (forma × color)
gen-catalogo.mjs	Arma los 400 productos de ejemplo desde 132 insumos
gen-imagenes.mjs	Dibuja las ilustraciones SVG de respaldo
gen-ubigeo.mjs	Compacta el ubigeo del INEI a public/ubigeo.json
gen-pdf.mjs	Arma el manual desde este README e imprime los tres PDF
verificar-imagenes.mjs	Comprueba que las ilustraciones no se corten
importar-fotos.mjs	Importa las fotos propias desde img/ (empareja por nombre)
traer-fotos.mjs	Descarga fotografías libres y guarda sus créditos
test/	512 tests con node:test (npm test)
public/	Tienda, panel, login, hoja de contactos, /img
img/	Las fotos originales del negocio, FUERA de public/
docs/	Los tres documentos, en HTML y PDF
data/	La base de datos (se crea sola)

El catálogo de 400 productos

Son productos inventados. Son creíbles, están bien categorizados y sirven para que la demo se vea llena y el asesor tenga con qué trabajar, pero **no son el catálogo real de ningún negocio.** Antes de operar hay que reemplazarlos, igual que las fotos de banco libre.

Por eso quedan **marcados en la base**, con la columna `productos.demo`. Los 24 curados entran con `demo = 0`, igual que todo lo que el dueño dé de alta desde el panel; el relleno generado entra con `demo = 1`. La marca hace una sola cosa, y es la que importa: **el relleno nunca se recomienda por delante de un producto del negocio**. En la tienda del cliente, con su catálogo cargado, no habrá ninguno marcado y el criterio se apaga solo.

La marca no sale en `/api/productos`: ordena por dentro, pero la tienda no va etiquetando productos de "demostración" en una respuesta que el cliente puede abrir en el navegador.

No están escritos a mano: `gen-catalogo.mjs` combina 132 insumos reales (maca, uña de gato, sangre de grado, propóleo...) con las presentaciones que le corresponden a cada uno. Así los nombres salen naturales —*Maca Negra en cápsulas*— y no aparecen disparates como *Jabón en gotas*. La semilla del generador es fija: dos ejecuciones dan exactamente el mismo catálogo.

El stock tampoco es al azar plano

Lo era —`entre(0, 60)`— contra un mínimo de 5 a 15— y con eso **uno de cada seis productos nacía en o por debajo de su mínimo**: el panel abría con **66 alertas de reposición**, una lista tan larga que el dueño no encuentra en ella lo que de verdad tiene que comprar. Una alerta que salta siempre no es una alerta.

Ahora se reparte en tres tramos:

TRAMO	CUÁNTO	STOCK
Agotado	1,5 %	<code>0</code>
Por reponer	2,5 %	entre 1 y el mínimo
Surtido	el resto	del mínimo + 5 hasta 60

Sobre los 400 quedan **19 por reponer, 6 de ellos agotados**. Es lo que se parece a un almacén de verdad: casi todo surtido, unos pocos en rojo y algún hueco. Y es lo que hace que los dos indicadores del guion signifiquen algo — el *Agotado* de la vitrina en el Acto 1, y el Propóleo marcado *1 de 10* en *Reposición urgente* del panel en el Acto 2.

Los 24 curados no pasan por aquí: su stock está escrito a mano en `db.js`, que es lo que fija las 4 unidades del propóleo y las 42 bolsas que hacen que la cotización mayorista del Acto 3 no alcance. Regenerar el catálogo no los toca.

Categorías: Hierbas, Superalimentos, Suplementos, Tónicos, Colágeno, Aceites, Esencias, Apícolas, Cremas y Cuidado personal.

`db.js --reset` carga primero los 24 productos curados —los que tienen foto real y el texto más cuidado— y completa hasta 400 con el catálogo generado, saltando nombres repetidos y corriendo los códigos que choquen.

Para cargar el catálogo real del negocio hay dos caminos: darlos de alta uno por uno desde el panel, o reemplazar `data/catalogo.json` por el listado real y correr `node db.js --reset`.

Alta de productos

Botón + **Nuevo producto** en el bloque *Catálogo* del panel. Lo que se valida en el servidor –no solo en el formulario:

DATO	REGLA
Nombre	3 a 90 caracteres, no puede repetirse
Categoría	Se sugieren las existentes; se admite una nueva
Precio	Mayor que cero
Costo	No puede superar al precio: sería cargar un producto que se vende a pérdida
Stock inicial	Entero. Entra al kardex como movimiento de <i>ingreso</i>
Código (SKU)	Se genera solo con la sigla de la categoría; se puede forzar uno
Imagen	Ruta del propio sitio (<code>/img/...</code>) o URL <code>https</code> . Nada más
Etiquetas	Se guardan sin tildes y en minúsculas, que es como llega la consulta del asesor

Un **tipo de producto nuevo** se crea escribiéndolo en el campo *Categoría* del alta –el desplegable sugiere las que ya existen, pero admite cualquier otra–. Tres cosas pasan solas:

- **Aparece como filtro propio** en la tienda y en la marquesina del inicio, que se arma con las categorías reales del negocio, no con una lista escrita a mano.
- **Recibe su propia ilustración.** Están dibujadas las 84 combinaciones de silueta y color, así que cualquier categoría nueva tiene lámina. El color sale del nombre de la categoría, de forma estable: *Mascotas* siempre será ámbar y no se verá igual que *Velas*.
- **Obtiene sigla para el código:** *Mascotas* → `MAS-001` .

Escribir «hierbas» no crea una categoría aparte de «Hierbas» –ni con tildes de por medio–: se adopta la grafía que ya estaba. Si de verdad es nueva, se guarda con la mayúscula inicial. Un catálogo partido entre «Hierbas», «hierbas» y «HIERBAS» es un filtro que miente.

Tres decisiones:

- **El margen se calcula mientras se escribe** y se pone rojo si es negativo. El dueño de una tienda no saca porcentajes de cabeza mientras atiende; ver el número bajando es lo que frena un precio mal puesto antes de guardarlo.
- **El stock inicial no aparece de la nada:** se registra como ingreso, igual que una reposición, para que el kardex explique cada unidad desde el día uno.
- **El alta queda firmada** en `cambios_producto` , con el usuario que la hizo.

El formulario insiste en las **etiquetas** porque son lo que hace que el producto aparezca cuando un cliente escribe «*me duele la cabeza*». Sin ellas el producto existe en la tienda, pero el asesor no lo va a ofrecer nunca.

Edición del catálogo

Desde el panel se puede cambiar **precio**, **stock mínimo** y **dar de baja o reactivar** un producto. Tres decisiones detrás:

- **El stock no se edita a mano.** Solo se mueve por ventas, ingresos y devoluciones, para que el kardex explique cada unidad. Para corregirlo se usa el ingreso de mercadería, que deja constancia.
- **Cada cambio queda firmado** en la tabla `cambios_producto` : qué campo, valor anterior, valor nuevo, quién y cuándo. Aparece en el bloque *Cambios de ficha*. Un precio que baja sin responsable es el mismo agujero que un ajuste de stock sin responsable.
- **Dar de baja no borra nada.** El producto desaparece de la tienda y el asesor deja de recomendarlo, pero conserva stock e histórico y se puede reactivar. El nombre de la columna a actualizar sale de una lista blanca del servidor, nunca del cuerpo de la petición.

Checkout peruano

El pedido pide y **valida en el servidor**:

DATO	VALIDACIÓN
DNI	8 dígitos → emite boleta
RUC	11 dígitos con dígito verificador módulo 11 → emite factura
Razón social	obligatoria con RUC
Correo	obligatorio con factura (es donde va el comprobante)
Teléfono	celular de 9 dígitos o fijo; normaliza +51 , espacios y guiones
Ubigeo	departamento / provincia / distrito contra los 1 874 distritos del INEI

La terna de ubigeo se valida anidada, no por separado: que "Cayma" exista no basta si dices que está en Lima. Ese es el error que deja direcciones imposibles en la base.

El envío lo calcula el servidor a partir del distrito, nunca el navegador:

Breña	→ Lima Centro	S/ 6.00	ubigeo 150105
Jesús María	→ Lima Moderna	S/ 9.00	ubigeo 150113
San Juan de L.	→ Lima Norte, Sur y Este	S/ 14.00	ubigeo 150132
Cusco/Wanchaq	→ agencia, flete en destino		ubigeo 080108

`POST /api/envio` cotiza antes de comprar, así el carrito muestra el costo real en vez de "según tu zona". Las zonas usan los **nombres oficiales del INEI**: si se escribe mal un distrito en `tienda.config.js` , esa zona nunca se aplicaría.

El validador de RUC detectó que el RUC de ejemplo que tenía la configuración (`20512345678`) **no era válido**. Ahora es `20512345671` , que sí pasa.

El código de pedido que se repetía

El código es `RA-AAAAMDD-XXX` : tres caracteres, porque se dicta por teléfono y se lee en voz alta en el mostrador. Eso son $36^3 = 46\,656$ **códigos por fecha**, y `pedidos.codigo` tiene índice único. Sin reintento, dos pedidos del mismo día que sacaran el mismo código acababan en `500 No se pudo registrar el pedido` . **La venta se perdía en la caja, por un código repetido.**

No es un caso de laboratorio, es el problema del cumpleaños:

PEDIDOS EN UN DÍA	PROBABILIDAD DE PERDER UNO
40	1,7 %
80	6,3 %
150	22 %
400	82 %

Ahora el `INSERT` va en un bucle dentro de la misma transacción: en SQLite un `UNIQUE` aborta la sentencia, no la transacción, así que reintentar con otro código no obliga a reabrir nada. Ocho intentos – con 400 pedidos ya cargados en la fecha, que fallen los ocho es 3 entre 10^{17} .

Y el azar dejó de salir de `Math.random()` . El código es una **credencial débil** – abre el seguimiento junto con los últimos cuatro dígitos del teléfono– y `Math.random` es predecible a partir de unas pocas salidas: quien hiciera dos pedidos podía adivinar los ajenos. Ahora sale de `node:crypto` .

El código sigue teniendo tres caracteres. Alargarlo habría tapado la colisión, pero el código existe para dictarse por teléfono; el reintento no se le nota a nadie y el largo tampoco.

La prueba que lo cubre ocupa un cuarto del espacio de la fecha por debajo de la API y mete 25 pedidos seguidos: contra el código anterior se caía en el tercero.

Comprobantes electrónicos

Al registrar la venta se emite el comprobante que corresponde:

DOCUMENTO DEL CLIENTE	COMPROBANTE	SERIE
DNI	Boleta de venta electrónica (<code>03</code>)	<code>B001</code>
RUC	Factura electrónica (<code>01</code>)	<code>F001</code>
Anular o devolver	Nota de crédito (<code>07</code>)	<code>BC01</code> / <code>FC01</code>

Cada comprobante trae número correlativo, desglose de IGV, importe en letras, contenido del QR y **XML UBL 2.1** descargable. La representación impresa está en `/comprobante.html?id=...` , se imprime en A4 y **se descarga en PDF**.

Lo que hace y lo que no:

✓ Numeración correlativa sin huecos	índice único (serie, correlativo) en la base
✓ Cálculo de IGV cuadrado al céntimo	probado con las cantidades que fuerzan redondeo
✓ Códigos de catálogo SUNAT	tipo doc, identidad, moneda, afectación, tributo, unidad
✓ XML UBL 2.1 con el hueco de la firma	ext:ExtensionContent vacío, listo para firmar
✓ Notas de crédito por anulación y devolución	motivos 01 y 06
✗ Firma digital	necesita el certificado .pfx del contribuyente
✗ Envío a SUNAT / OSE	necesita usuario SOL secundario y endpoint del OSE
✗ CDR	lo devuelve SUNAT al recibir el comprobante firmado
✗ QR gráfico	el contenido está; dibujarlo necesita una librería

Los comprobantes quedan en estado `pendiente_envio` : **emitidos y numerados, pero sin CDR**. Presentarlos como aceptados por SUNAT sería mentir. La función `enviarASunat()` en `comprobantes.js` está vacía a propósito y documenta las tres cosas que hacen falta para completarla.

La boleta en PDF, sin librerías

La representación impresa vivía solo como página web: para mandársela al cliente por WhatsApp había que enviarle un enlace o una captura de pantalla. Ahora se descarga como PDF – la tienda desde el panel, y **el comprador desde su propia página**, con su código y los últimos cuatro dígitos del teléfono. Nadie tiene que llamar al local a pedir su boleta.

El escritor está en `pdf.js`, a mano y sin dependencias, como todo lo demás. Un PDF es texto plano con una tabla de posiciones al final: se puede escribir si uno se limita a lo que hace falta –texto, líneas y rectángulos– y usa las catorce tipografías *base 14* que todo lector trae incorporadas. Al no incrustar tipografías, **un comprobante pesa unos 4 KB en vez de 300**, que en algo que se manda por WhatsApp con datos móviles importa. `comprobante-pdf.js` es el que dibuja la boleta encima de ese escritor.

El archivo sale con el nombre que pide SUNAT –RUC, tipo de documento, serie y correlativo:

```
20512345671-03-B001-00000001.pdf
```

No es capricho: es lo que hace que los comprobantes de todo un mes se ordenen solos en la carpeta del contador.

Un bug que valía dos céntimos

La primera versión calculaba `valorUnitario = redondear(precio / 1,18)` y luego multiplicaba por la cantidad. Ese doble redondeo hacía que $3 \times S/ 22,00$ diera $S/ 65,99$, y el comprobante salía **dos céntimos por debajo de lo cobrado**. Un comprobante que no cuadra con el cobro lo rechaza SUNAT y descuadra la caja.

Ahora el ancla es el **importe** –lo que el cliente paga– y de ahí se derivan la base imponible y el IGV por diferencia. El valor unitario se guarda con diez decimales, que es exactamente para lo que SUNAT admite ese margen.

Acceso al panel

Lo que se protege no es "el panel" en abstracto: es **GET /api/pedidos**, que devuelve **nombre, teléfono y dirección de cada cliente que compró**. Dejarlo abierto no era un descuido de demo, era una fuga de datos personales.

RUTA	SIN SESIÓN
/ , /creditos.html , /api/tienda , /api/productos , /api/asesor , POST /api/pedidos	✅ abiertas – es la tienda
GET /api/pedidos , /api/admin/* , POST /api/stock	401
/admin.html , /imagenes.html	302 → /login.html

Cómo está hecho, sin dependencias:

- **Claves con `scrypt` (`node:crypto`)**, salt de 16 bytes por usuario y comparación en **tiempo constante** – no filtra cuánto acertó un atacante.
- **Sesión en cookie `HttpOnly` + `SameSite=Strict`**, token de 32 bytes guardado en la tabla `sesiones`, caducidad de 8 horas (una jornada del local). `SameSite=Strict` es lo que cubre el CSRF sin token aparte.
- **Bloqueo por fuerza bruta**: 5 intentos fallidos → 5 minutos de espera, incluso si luego aciertan la clave. Se guarda en memoria: reiniciar limpia los bloqueos.
- **Las cuotas por IP usan la IP de la conexión**, no la cabecera `X-Forwarded-For`: esa la escribe el cliente, y rotarla anulaba los límites de login, asesor y seguimiento. Detrás de un proxy propio (nginx, Caddy) que la reescribe, se activa con `CONFIAR_PROXY=1`; sin eso, todos los clientes compartirían la IP del proxy.
- **El error de login nunca distingue** entre usuario inexistente y clave mala.
- **Cambiar la clave cierra las sesiones abiertas** de esa persona.
- **El kardex anota quién fue**: un ajuste manual queda como `Compra a proveedor Oxapampa (rosa)`. Un movimiento de stock sin responsable es justo por donde se pierde mercadería sin que nadie lo note.

Al publicar con HTTPS, agrega `Secure` a la cookie en `auth.js` (`cookieSession`).

Falta a propósito porque la demo corre en `http://localhost`.

Las tres capas del asesor

Probando la tienda como cliente real, **8 de 8 preguntas que no eran "recomiéndame un producto" terminaban en un catálogo**. La peor: *"quiero devolver el propóleo, no me hizo nada"* respondía ofreciendo el mismo propóleo.

Por eso el asesor tiene tres capas, en este orden:

CAPA	QUÉ ATIENDE	DE DÓNDE SACA LA RESPUESTA
1. Cotización	"50 bolsas de maca negra"	precio + escala de descuento + si alcanza el stock (sin decir la cifra)
2. Intenciones	saludo, despedida, presencia, catálogo, WhatsApp, delivery (Lima y provincia), horario, pago, comprobante, ubicación, regateo, mayorista, devolución, reclamo, autenticidad	<code>tienda.config.js</code> y, en un reclamo con código, la tabla <code>pedidos</code>
3. Productos	síntomas y necesidades	catálogo filtrado contra stock real

La comparación – "¿qué es mejor, la maca o el camu camu?" – no pasa por la tabla de intenciones: se detecta en `asesor.js` sobre los nombres del catálogo, porque necesita saber de qué dos productos habla.

Seis decisiones que valen la pena señalar:

- **El regateo no se improvisa.** No responde con un descuento inventado, sino con la escala de `tienda.config.js` (10 / 15 / 20 % según cantidad). El asesor nunca regala margen del negocio.
- **Una cotización siempre dice si alcanza.** Es el argumento de venta completo en una sola respuesta: precio correcto y si se entrega todo hoy o una parte después, en el pedido donde más cuesta equivocarse. La cifra no sale: el inventario es información del negocio, no de quien pregunta.
- **El asesor no cambia el estado de un pedido.** Informa y compromete a la tienda, pero no ejecuta. Si lo hiciera, cualquiera con un código movería el inventario desde el chat.
- **Un saludo no dispara un catálogo.** `hola` y `ok gracias` están anclados con `^...$`, así que "hola, tienes algo pa dormir" sigue de largo a productos.
- **El relleno de demostración va último, empate o no.** Un producto inventado con las etiquetas perfectas no desplaza a uno que el negocio sí tiene, por muchas señales que acierte. Ver abajo.
- **El asesor no receta.** Ante una enfermedad, un medicamento, un embarazo, un bebé o un síntoma de alarma no recomienda nada y ofrece atención con la dueña. Ver *El asesor no receta*, a continuación.

Cómo escribe la gente

El asesor se midió contra un banco de **146 consultas reales**, escritas como llegan por WhatsApp: "q tienen pa la tos d mi hijo", "a q numero yapeo", "toy embarazada", "uña de gat". Antes acertaba el **76,7 %**; hoy el 100 % de ese banco. Con **36 frases de control** que no se usaron para ajustarlo acertó el **83,3 %** antes de corregir lo que destaparon, y esa es la cifra honesta de cuánto generaliza.

Cuatro piezas, todas sin dependencias:

- **Jerga de WhatsApp a diccionario** (`expandirJerga`): *q, x, pa, d, toy, bb, xfa...* y las letras repetidas (*holaaa*). Por palabra entera y con letras Unicode: `\b` de JavaScript cree que la «é» corta la palabra.
- **Erratas de producto** (`corregirErratas`): *maka, hercanpuri, propolio*, corregidas **contra el vocabulario del propio catálogo** y solo con un candidato claro. Nunca se corrigen palabras comunes (*cada* estaba a una letra de *caída*) ni las que el asesor ya entiende.

- **La derivación no depende de la ortografía:** fonética de oído (v/b, z/s, ce/se) y una letra de distancia en palabras largas y específicas. *Embarasada* y *diabetis* derivan igual. La corrección de erratas **no** corre antes de derivar: una enfermedad mal escrita no puede «corregirse» a un producto.
- **Intenciones con variantes reales:** *yapeo*, *rebajita*, *me llegó roto*, *ya pasaron 3 días y nada*, *es bamba*.

Y dos cambios de tono. La respuesta cierra con una pregunta útil –¿es para ti o para alguien de la casa?, o ¿lo recoges en el puesto o te lo mandamos? si ya nombró el producto– y, cuando no entiende, **pregunta qué busca en vez de ofrecer tres productos fijos.**

`test/asesor-entrenamiento.test.mjs` guarda una muestra del banco para que no se desentrene.

El asesor no receta

Un suplemento o una hierba no puede promocionarse como que previene, trata o cura una enfermedad, y decir qué tomar y cuánto es una posología. Un asistente que recibe un diagnóstico y devuelve un producto hace exactamente eso, por escrito y guardado en un servidor. Por eso la derivación va **antes** que todo lo demás (`hayQueDerivar()` en `asesor.js`).

La primera versión solo reconocía la palabra técnica, y la gente no habla así. Probado contra la tienda en marcha:

CONSULTA	ANTES	AHORA
"tengo el azúcar alta"	Jarabe de Yacón · Yacón en hojuelas	deriva
"mi bebé tiene tos"	Miel de Abeja · Propóleo (miel antes del año: riesgo de botulismo)	deriva
"tengo dolor de pecho"	Aceite de Copaiba · Uña de Gato	deriva
"tomo sertralina y estoy con estrés"	Valeriana + Pasiflora · Magnesio	deriva

La lista cubre ahora cinco frentes: embarazo y lactancia dichos de cualquier forma ("espero un bebé", "estoy lactando"), bebés y edad en meses, **síntomas de alarma** (pecho, falta de aire, desmayo, palpitaciones, sangrado, fiebre), crónicas dichas como las dice el cliente ("problemas de presión", "piedras en la vesícula") y **cualquier medicamento**, por su nombre genérico o como "tomo pastillas para..." / "me recetaron...".

Los términos nuevos traían sus propios parecidos, y cada uno tiene su test: "cómo se bebe la muña" (el verbo), "algo para beber", "sin lácteos", "propóleo en gotas" y "venden antibióticos" (sin tomarlos sigue siendo «no trabajamos medicamentos de farmacia») **no** derivan.

Lo que **no** deriva a propósito: descanso, digestión, energía, ánimo, defensas, cuidado de la piel. Son categorías de bienestar y el asesor las atiende. Si alguien pregunta **cuánto** tomar ("cuánta muña tomo al día"), no se le da una cifra: la respuesta manda a la forma de uso del envase del fabricante.

El catálogo tampoco promete curar. La ficha se ve en la tienda aunque el asesor derive, y es el texto que se le pasa al modelo. Los 400 productos traían "Cicatrizante heridas y calma la gastritis", "Endulzante apto para diabéticos", "Circulación, próstata y anemia" y posologías como "Cucharada en ayunas" o "Dos veces al día", además de etiquetas como `colesterol` , `artritis` o `presion` . Se reescribieron en `gen-catalogo.mjs` y `db.js` como bienestar

y uso tradicional –historia del producto, no promesa de efecto–, sin tocar precios, stock ni códigos. `test/catalogo-sin-claims.test.mjs` revisa el catálogo generado y la semilla, y falla si vuelve a entrar una enfermedad o una dosis.

Al cargar el catálogo real del negocio (CSV o panel), esa revisión no corre sola: los textos del proveedor suelen traer claims. Conviene pasar el test sobre la base cargada antes de publicar.

Dos magnesios que tapaban a la valeriana

A "no puedo dormir y ando con mucho estrés" el asesor contestaba así:

	PRODUCTO	ORIGEN
1	Magnesio Quelado efervescente	Formulado en Lima
2	Magnesio en polvo	Formulado en Lima
3	Valeriana + Pasiflora	Sierra sur

Los tres empataban a puntos –las mismas cuatro señales: *dormir*, *insomnio*, *estrés*, *ansiedad*–, y el desempate era **el stock**: 39 y 35 contra 14. Ganaba el relleno generado.

Dolía en el peor sitio. El Acto 1 vende que "el origen es el argumento –Meseta de Bombón, Junín, no solo Maca", y dos pantallas después el propio asesor abría con "**De Formulado en Lima**", dos veces seguidas, en la respuesta que cierra la demo. El sistema desmentía el argumento de venta.

Y encima se colaban los dos juntos, que resultó ser un problema aparte y más de fondo.

La misma planta tres veces

`variar()` existe justo para que no salga el mismo insumo tres veces, pero comparaba las raíces **por igualdad**. Donde peor se veía era en las rodillas:

"me duelen las rodillas" → **Uña de Gato · Crema de Uña de Gato · Tónico de Uña de Gato**

Las tres son entradas **separadas** del catálogo, cada una con su categoría, su origen y su precio: una cápsula, una crema y un tónico. Para el sistema eran tres productos distintos, y lo son. Para quien preguntó qué tomar para la rodilla son uña de gato tres veces. Lo mismo pasaba con *Magnesio* y *Magnesio Quelado*, que tampoco son una el envase de la otra: son dos fichas del catálogo.

Ahora dos raíces son el mismo insumo si **una contiene a la otra entera**, por palabras completas: `uña de gato` está dentro de `crema de uña de gato`, y `magnesio` dentro de `magnesio quelado`. Palabras enteras a propósito, para no casar a media palabra.

Y se recortan además las presentaciones que van **delante** del insumo –*Crema de*, *Tónico de*, *Esencia de*, *Aceite de*...–, con lista blanca: *Cola de Caballo*, *Miel de Abeja*, *Diente de León* y *Sangre de Grado* empiezan igual y no son envases. Recortarlas dejaría raíces absurdas como *caballo* o *grado*.

Sobre los 400 productos, eso los agrupa en **122 insumos**. El resultado:

CONSULTA	ANTES	AHORA
"me duelen las rodillas"	Uña de Gato · Crema de Uña de Gato · Tónico de Uña de Gato	Uña de Gato · Cúrcuma con Pimienta · Colágeno con Magnesio
"algo para los nervios"	Valeriana + Pasiflora · Pasiflora en filtrantes	una sola pasiflora

("tengo gastritis" también repetía planta; hoy ya no recomienda nada: deriva, ver *El asesor no receta.*)

El test destapó uno más que no habíamos visto: "algo para los nervios" ofrecía **Valeriana + Pasiflora** y **Pasiflora en filtrantes** en la misma respuesta.

Ahora **demo** es el **primer** criterio de orden, antes que los puntos:

	PRODUCTO	ORIGEN
1	Valeriana + Pasiflora	Sierra sur
2	Manzanilla Orgánica	Valle del Mantaro
3	Graviola en cápsulas	Amazonía

No es un desempate, es una precedencia, y es a propósito: el relleno es **inventado**.

Recomendar un producto que no existe por delante de uno que está en el almacén es responder mal, aunque coincidan más etiquetas. Su trabajo es que el estante se vea lleno, no vender.

El cotizador rechaza más de lo que acepta

Un número en un mensaje casi nunca es una cantidad de compra. Estos cuatro casos reales cotizaban de más antes de los filtros:

MENSAJE	COTIZABA	AHORA
"la maca de 250 gramos"	250 bolsas, S/ 6 900	va a productos
"el propóleo de 30 ml"	30 frascos	va a productos
"me lo dejas en 20 soles"	20 macas	responde el regateo
"tomo 2 pastillas al día"	2 macas	no vendemos medicamentos

Para cotizar se exigen dos cosas: que la cifra **no** vaya seguida de una unidad de medida, edad o precio (**ml** , **gramos** , **años** , **soles**), y que haya intención de compra – un envase pagado a la cifra ("50 bolsas") o un verbo ("quiero", "necesito", "cuánto me sale").

Imágenes: tres niveles de respaldo

Cada tarjeta intenta cargar, en este orden:

NIVEL	ORIGEN	PESO
1. Fotografía	public/img/fotos/ – propias del negocio, o de bancos libres	2,5 MB total
2. Ilustración	public/img/<SKU>.svg – generada localmente	~2 KB c/u

NIVEL	ORIGEN	PESO
3. Marca	<code>public/img/placeholder.svg</code>	1 KB

El navegador no propaga el evento `error` de una imagen hacia arriba, pero sí se puede capturar en la fase de captura: **un solo oyente cubre todas las imágenes**, incluidas las que aún no existen cuando se registra. Nunca se ve un cuadro roto.

Las fotos se descargan, no se enlazan. Enlazar deja la tienda dependiendo de un CDN ajeno: sin internet en el local, la demo se llena de huecos grises justo en la pantalla que estás presentando.

Por qué Openverse y no Pixabay: Pixabay responde 403 a peticiones automatizadas, y sobre todo Openverse devuelve **la licencia y el autor** de cada imagen – que es lo que el cliente necesita para publicar sin problemas legales. Se piden CC0 y dominio público primero. Hoy las 24 fotos se reparten así:

ORIGEN	CUÁNTAS	ATRIBUCIÓN
Propias del negocio (<code>importar-fotos.mjs</code>)	12	no hace falta
CC0 y dominio público	8	no hace falta
CC BY	4	obligatoria

Por esas 4 existe `/creditos.html`, enlazada en el pie. A medida que el cliente entregue sus fotos, la columna de CC BY se vacía y la página deja de hacer falta.

Estas fotos son marcadores de posición. Antes de publicar hay que reemplazarlas por las del propio negocio – no son los productos reales del puesto.

Para usar las fotos del cliente basta la columna `imagen`, que acepta cualquier URL o ruta local:

```
UPDATE productos SET imagen = '/img/fotos/mi-maca.jpg' WHERE sku = 'MAC-001';
```

`node verificar-imagenes.mjs` comprueba que ninguna ilustración se salga del lienzo ni quede descentrada – útil cuando no puedes abrir el navegador para mirarlas.

La decisión de diseño que sostiene la demo

El motor de reglas elige los productos; la IA solo redacta.

`asesor.js` filtra el catálogo contra el stock real y puntúa por etiquetas y sinónimos coloquiales ("*no puedo dormir*", "*me duelen las rodillas*", "*chuchaqui*"). Recién esa lista ya validada pasa al modelo para que escriba la respuesta.

Consecuencias, todas buenas para una demo frente a un cliente:

- **Nunca recomienda un producto agotado**, aunque el modelo alucine.
- **Funciona sin internet y sin API key** – cae a una respuesta por plantilla.
- **Es barato:** una llamada corta por consulta, no un agente conversacional.

Activar la redacción con Claude (opcional)

```
npm install @anthropic-ai/sdk
set ANTHROPIC_API_KEY=sk-ant-... # PowerShell: $env:ANTHROPIC_API_KEY="sk-ant-..."
npm start
```

Sin esas dos cosas la tienda funciona igual, con el motor de reglas. El campo `fuelle` de `/api/asesor` dice `"reglas"` o `"claude"`.

Modelo: `claude-opus-5`, `effort: low` (respuestas cortas y rápidas). El prompt del sistema le prohíbe atribuir propiedades curativas, nombrar enfermedades, indicar dosis y dar cifras de stock – requisito legal para publicidad de productos naturales. Y aunque el modelo se saltara la regla, no elige productos: la derivación y la lista vienen resueltas del motor de reglas.

API

MÉTODO	RUTA	QUÉ HACE
POST	/api/login	{correo, clave} → cookie de sesión (acepta usuario por compatibilidad)
POST	/api/logout	Cierra la sesión
GET	/api/sesion	Quién está conectado, o 401
GET	/api/tienda	Datos públicos: envío gratis, horario, pagos
POST	/api/envio	Cotiza el envío de un distrito antes de comprar
GET	/api/productos	Catálogo activo
GET	/api/productos/:id	Un producto suelto
POST	/api/asesor	{consulta} → intención de negocio, o recomendaciones filtradas por stock
POST	/api/pedidos	Registra pedido y descuenta stock en una transacción
GET	/api/pedidos	 Pedidos con sus ítems (trae datos del cliente)
PATCH	/api/pedidos/:id/estado	 Avanza estado; <code>anulado</code> devuelve el stock
POST	/api/stock	 Reposición o ajuste manual, con motivo y responsable
GET	/api/admin/resumen	 KPIs, bajo stock, más vendidos
GET	/api/admin/movimientos	 Kardex de las últimas 60 operaciones
GET	/api/admin/productos	 Catálogo completo, incluidos los dados de baja
GET	/api/admin/cambios	 Bitácora de cambios de ficha
GET	/api/admin/categorias	 Categorías existentes, para el alta
POST	/api/productos	 Alta de producto; el stock inicial entra al kardex

MÉTODO	ruta	QUÉ HACE
PATCH	/api/productos/:id	 Cambia precio, stock mínimo o alta/baja
POST	/api/seguimiento	Estado del pedido con código + últimos 4 del teléfono
POST	/api/seguimiento/recibido	Constancia de recepción del cliente
GET	/api/admin/comprobantes	 Comprobantes emitidos
GET	/api/comprobantes/:id	 Uno, con su desglose de IGV
GET	/api/comprobantes/:id/pdf	 Representación impresa en A4
GET	/api/comprobantes/:id/xml	 XML UBL 2.1, con el hueco de la firma
POST	/api/seguimiento/comprobante	Su boleta o factura, para imprimir. Sin el XML
GET	/api/seguimiento/comprobante/pdf	El PDF del comprador, con <code>?codigo=</code> y <code>?tel=</code>

 = requiere sesión.

El pedido entra completo o no entra: si un solo ítem no tiene stock, se revierte todo y se devuelve `409` con el detalle de qué faltó.

Lo que falta para producción

Honestidad con el cliente sobre el alcance de esta demo:

1. **HTTPS y cookie `Secure`** – obligatorio antes de publicar fuera de la laptop.
2. **Pasarela de pago** – el pedido queda registrado, el cobro es contra entrega.
3. **Respaldo de la base** – copiar `data/tienda.db` a diario.
4. **Fotos reales de producto** – 12 de los 24 productos con foto ya son del negocio; las otras 12 son de banco libre y el resto del catálogo va con ilustración. Las que falten entran con `importar-fotos.mjs`.
5. **Firma digital y envío a SUNAT** – los comprobantes se emiten, se numeran y se descargan en PDF y XML, pero quedan en `pendiente_envio`: falta el certificado `.pfx` del contribuyente, el usuario SOL secundario y el endpoint del OSE. Es rellenar `enviarASunat()`, un archivo.